Frontend Engineering Challenge

National Service Personnel (NSP) Technical Assessment Series • FleetPulse Console

DURATION: 72 Hours (3 Days)	**EST. EFFORT:** 14–18 Hours	**ROLE:** Frontend Engineer (React)
FOCUS / STACK: React / TypeScript / Web Workers	**DELIVERABLE:** GitHub Repo / Deployed Link	**LEVEL:** NSP Assessment

1. Project Brief & Business Problem

A regional logistics fleet management company operates hundreds of active transit assets equipped with IoT telemetry modules. Fleet dispatchers monitor vehicle locations, fuel/battery health, operational statuses, and critical emergency alerts in real time.

The company's legacy internal application regularly freezes or crashes when processing high-volume live data streams, causing dispatchers to miss critical alerts (e.g., battery depletion, unauthorized route deviation). You are tasked with building the next-generation FleetPulse Telemetry Console, optimized for heavy data throughput, zero UI lag, and deep state resilience.

2. Functional & Technical Requirements

A. High-Frequency Real-Time Streaming & State Batching

- **Mock Telemetry Generator:** Implement a Web Worker or WebSocket/SSE simulator broadcasting telemetry updates for 1,000+ assets at a rate of 2–5 events per second.
- **Render Performance:** Implement state batching and UI debouncing to prevent excessive DOM re-renders. The UI frame rate must stay locked at ~60 FPS during stream ingestion.
- **Virtualization:** The central asset list/table must utilize DOM virtualization (react-window or TanStack Virtual) to smoothly scroll through thousands of dynamic items without degradation.

B. Multi-Dimensional Filtering & URL Synchronization

- **Filtering Controls:** Provide instant search (debounced), status filtering (Active, Idle, Maintenance, Offline), battery level range sliders, and tag filters.
- **Deep-Linking:** Bidirectionally sync all active filter, search, sorting, and pagination parameters with the browser URL query string (URLSearchParams). Refreshing or sharing the URL must restore the exact view state.

C. Optimistic UI Updates & Error Rollback

- **Remote Actions:** Dispatchers can trigger high-priority remote actions: Lock Vehicle, Reroute Asset, or Dispatch Maintenance.
- **Optimistic Mechanics:** Apply optimistic state updates immediately upon user click. Simulate a 20% network failure rate on command requests. If the mock server returns an error, automatically roll back the UI state to its pre-action baseline and display a toast alert.

D. Session Persistence & Offline Resilience

- **Local Storage:** Persist incoming telemetry data and active session settings locally using IndexedDB or localStorage.

- **Offline Mode:** If network connection is lost, transition to an 'Offline Mode' banner while retaining full offline inspection and search capability over cached assets.

3. Architectural & Testing Mandates

- **TypeScript:** Strict TypeScript mode enabled (noImplicitAny: true). All telemetry payloads, filter states, and action dispatchers must be strongly typed.
- **State Management:** Clean state modularity (React Context, Zustand, Redux Toolkit, or TanStack Query).
- **Testing:** Unit tests covering the filter pipeline, URL query parser, and optimistic rollback state machine using Vitest or React Testing Library. Minimum 70% test coverage required.

4. Deliverables & Evaluation Criteria

Required Submission Items

1. Public GitHub Repository containing clean, atomic git commits.
2. Comprehensive README.md outlining architecture choices, performance benchmarks, and local setup instructions.
3. Deployed Live Preview URL (e.g., Vercel, Netlify, or Cloudflare Pages).